

MoE Routing Latency Sweep

Study Goals

This is a characterization study. The goal is to measure how fused MoE kernel latency responds to each of three knobs, independently and jointly, across two representative kernel dimension configurations.

Knobs under investigation:

1. Token-to-expert routing distribution (imbalance ratio α)
2. Number of input tokens (batch size)
3. TP / EP degree

Scope: direct kernel invocation only. No vLLM serving pipeline, no tokenizer, no scheduler, no attention layer, no KV cache. The kernel receives pre-constructed input tensors and the gating network is bypassed entirely. Timings are CUDA event measurements around the kernel call itself.

Questions this study will answer:

- At what token count does each model's expert GEMM transition from memory-bound to compute-bound, and how does TP/EP shift that crossover?
- How does routing imbalance affect total kernel latency, and at what imbalance ratio does it dominate over compute time?
- How do DeepEPHT and DeepEPLL compare on latency and variance across the same input space?

Kernel Configurations

The kernel only sees weight tensors and token tensors — no model config, no tokenizer. Each configuration is defined entirely by expert dimensions and routing parameters.

Config name	Num experts (E)	Top-k	Hidden dim (d)	Expert intermediate dim (d_ff)	Weight size (2 experts × W1+W2, FP16)
<code>mixtral-7b-dims</code>	8	2	4096	14336	$2 \times (4096 \times 14336 + 14336 \times 4096) \times 2B \approx 938 \text{ MB}$
<code>mixtral-22b-dims</code>	8	2	6144	16384	$2 \times (6144 \times 16384 + 16384 \times 6144) \times 2B \approx 1.5 \text{ GB}$
<code>qwen2-57b-dims</code>	64	8	3584	2048	per GPU at EP=8: $8 \times (3584 \times 2048 + 2048 \times 3584) \times 2B \approx 470 \text{ MB}$

Weight tensors are initialized with `torch.randn` in the specified dtype — no checkpoint loading required. This keeps the benchmark self-contained and removes any HuggingFace / safetensors dependency.

The gating network is bypassed entirely: `topk_ids` and `topk_weights` are constructed programmatically (see routing section below).

Setup

Hardware (parameterized — fill in before running)

```

GPU_TYPE=          # e.g., H100 SXM5 80GB
NUM_GPUS=          # e.g., 8
INTERCONNECT=      # NVLink | InfiniBand | PCIe
NVLINK_BW=         # GB/s per direction, e.g. 450 for NVLink4
PCI_E_GEN=         # 4 | 5 (for cross-NUMA or PCIe-only topologies)
HOST_DRAM=         # GB (for KV tiering experiments in later phases)
CXL_ATTACHED=      # yes | no

```

Minimum viable config for EP > 1: **4 GPUs** (covers Mixtral 8×7B at EP=4, TP=1 or EP=2, TP=2).

Minimum for Qwen2-57B-A14B EP study: **8 GPUs** (EP=8, 8 of 64 experts per GPU, top-8 routing).

Kernel Backends

vLLM's FusedMoE Modular Kernel, invoked directly (not through the serving stack). Two backends under study:

- **DeepEPHT** — high-throughput all-to-all; optimized for large num_tokens, sustains bandwidth at cost of per-call latency
- **DeepEPPL** — low-latency all-to-all; optimized for small num_tokens, minimizes dispatch overhead at cost of throughput

Harness

vLLM modular kernel — invoked directly, not through the serving stack:

```

from vllm.model_executor.layers.fused_moe import FusedMoEModularKernel
from vllm.model_executor.layers.fused_moe.prepare_finalize import DeepEPHT # or DeepEPPL

kernel = FusedMoEModularKernel(prepare_finalize=DeepEPHT(...), experts=BatchedTritonExperts(...))

hidden_states = torch.randn(num_tokens, hidden_dim, dtype=torch.float16, device="cuda")
topk_ids, topk_weights = make_routing(num_tokens, num_experts, topk, mode=routing_mode)

# --- Deferred-event timing (no synchronize in the hot loop) ---
# Pattern from: github.com/gt-nexs/lynx/blob/main/src/lynx/metrics/metric_logging.py
#
# Record start/end events during every rep without blocking.
# Accumulate (start, end) pairs. Synchronize once after all reps.
# Call elapsed_time() only then – avoids a CPU-GPU sync per iteration.

NUM_WARMUP = 50
NUM_REPS = 100

for _ in range(NUM_WARMUP):
    kernel(hidden_states, w1, w2, topk_ids, topk_weights)

```

```

event_pairs = []
for _ in range(NUM_REPS):
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    kernel(hidden_states, w1, w2, topk_ids, topk_weights)
    end.record()
    event_pairs.append((start, end))

torch.cuda.synchronize() # single sync after all reps
latencies_ms = [s.elapsed_time(e) for s, e in event_pairs]
# report: mean, p50, p95, p99

```

The `profile_modular_kernel.py` in `tests/kernels/moe/` is useful for a first-pass PyTorch trace; the loop above is the production timing harness for the sweep.

Environment:

```

pip install torch triton
pip install vllm --no-deps # kernel + Triton deps only, no server

```

Methodology

Independent Variables

1. Routing Distribution (primary variable)

Parameterized by **imbalance ratio** $\alpha = \text{max_tokens_any_expert} / \text{avg_tokens_per_expert}$.

For batch size `B` with top-`k` routing and `E` experts:

- $\text{avg} = B \times k / E$
- $\alpha = 1.0 \rightarrow$ perfectly uniform (lower bound on dispatch latency)
- $\alpha = k \times E \rightarrow$ all tokens to one expert (upper bound; only possible if capacity unbounded)

Concrete routing distributions to test:

Name	Construction	Expected α range
<code>uniform</code>	Round-robin assignment; each expert gets exactly $\lfloor N \times k / E \rfloor$ or $\lceil N \times k / E \rceil$ tokens	1.0
<code>zipfian</code>	Expert k (ranked $1..E$ by popularity) receives probability $\propto 1/k^s$; default $s=1.0$. Most realistic proxy for observed MoE routing distributions.	2–6 $\times$ depending on E and s
<code>random</code>	Top- k drawn from Dirichlet(concentration=1.0) — mild natural imbalance	1.0–3.0
<code>skewed-2x</code>	One expert receives 2 $\times$ average; remainder share the rest uniformly	2.0
<code>skewed-4x</code>	One expert receives 4 $\times$ average	4.0
<code>worst-case</code>	All top- k slots assigned to k experts; remaining $E-k$ experts receive 0	N / avg

Generate `topk_ids` programmatically:

```

def make_routing(num_tokens, num_experts, topk, mode, zipf_s=1.0, skew_alpha=1.0):
    """

```

```

Returns topk_ids: [num_tokens, topk] int32
        topk_weights:[num_tokens, topk] float32 (uniform weights – isolates routing shape)
"""
if mode == "uniform":
    # Deterministic: each expert gets exactly floor or ceil tokens
    slots = (list(range(num_experts)) * ((num_tokens * topk) // num_experts + 1))[:num_tokens * topk]
    random.shuffle(slots)
    topk_ids = torch.tensor(slots, dtype=torch.int32).reshape(num_tokens, topk)

elif mode == "zipfian":
    #  $P(\text{expert } k) \propto 1/k^s$ , experts ranked 1..E by popularity
    ranks = torch.arange(1, num_experts + 1, dtype=torch.float32)
    probs = (1.0 / ranks ** zipf_s)
    probs /= probs.sum()
    topk_ids = torch.multinomial(probs.expand(num_tokens, -1), topk, replacement=False)

elif mode == "random":
    # Dirichlet(1) – mild natural imbalance
    probs = torch.distributions.Dirichlet(torch.ones(num_experts)).sample()
    topk_ids = torch.multinomial(probs.expand(num_tokens, -1), topk, replacement=False)

elif mode in ("skewed-2x", "skewed-4x"):
    # One hot expert gets skew_alpha * avg; rest share uniformly
    avg = num_tokens * topk / num_experts
    hot_count = min(int(skew_alpha * avg), num_tokens * topk)
    # ... assign hot_count slots to expert 0, distribute remainder uniformly
    topk_ids = _build_skewed(num_tokens, num_experts, topk, hot_count)

elif mode == "worst-case":
    # All slots go to the first topk experts
    topk_ids = torch.arange(topk, dtype=torch.int32).expand(num_tokens, -1).contiguous()

topk_weights = torch.ones(num_tokens, topk, dtype=torch.float32) / topk
return topk_ids.to("cuda"), topk_weights.to("cuda")

```

2. Number of Input Tokens

The kernel sees a flat `[num_tokens, hidden_dim]` tensor — no notion of batch or sequence length. "Batch size" in the context of decode mode corresponds directly to `num_tokens`. For prefill, `num_tokens = batch × seq_len`, but the kernel does not distinguish the two; sweep `num_tokens` as the raw variable.

Sweep (log-spaced to cover decode through long agentic prefill):

```
[1, 4, 16, 64, 256, 1024, 4096, 16384, 65536, 131072, 200000]
```

Regime annotations:

num_tokens range	Inference context
1-128	Single-step decode, small batch
128-4096	Large-batch decode or short prefill
4096-32768	Standard prefill (4K-32K context)
32768-200000	Long-context / agentic prefill — primary motivation for extending sweep

Annotate each point with:

- $\text{avg_tokens_per_expert} = \text{num_tokens} \times \text{top_k} / \text{num_experts}$
- Expert GEMM AMI = $2 \times \text{avg_tokens_per_expert}$ (flops/byte, from [arithmetic-intensity-decode](#))

Predicted memory-bound → compute-bound crossover (at EP=1, ridge point ≈ 295 flops/byte for H100):

Config	Crossover $\text{avg_tokens_per_expert}$	Crossover num_tokens
<code>mixtral-7b-dims</code> (8 exp, top-2)	~148	~592
<code>mixtral-22b-dims</code> (8 exp, top-2)	~148	~592
<code>qwen2-57b-dims</code> (64 exp, top-8)	~148	~1184

Both Mixtral configs cross into compute-bound well below 4096 tokens. The 4K-200K range is squarely in the compute-bound regime for EP=1; the interesting question there is whether dispatch and combine cost grows fast enough to re-introduce a memory-bound character to total kernel time at high EP.

3. TP / EP Configuration

Test configurations (constrained to $\text{TP} \times \text{EP} \leq \text{NUM_GPUS}$):

Config	TP	EP	Total GPUs	Notes
<code>tp1-ep1</code>	1	1	1	Baseline; all experts on one GPU
<code>tp2-ep1</code>	2	1	2	TP only; no all-to-all
<code>tp1-ep2</code>	1	2	2	EP only; all-to-all between 2 GPUs
<code>tp1-ep4</code>	1	4	4	EP=4; each GPU holds E/4 experts
<code>tp2-ep2</code>	2	2	4	Mixed; all-to-all + all-reduce
<code>tp1-ep8</code>	1	8	8	Full EP; 1 expert/GPU for Mixtral
<code>tp4-ep2</code>	4	2	8	TP-heavy; small expert shards

For Qwen2-57B-A14B (64 experts): EP=8 means 8 experts/GPU. EP=16 needs 16 GPUs (skip if unavailable).

Expert placement caveat: this study does not control which expert is assigned to which GPU — the default placement is assumed (experts 0..E/EP-1 on GPU 0, etc.). In practice, placement policy interacts with routing distribution: a Zipfian routing pattern with hot experts co-located on the same GPU produces different all-to-all traffic than one with hot experts spread across GPUs. This study treats placement as fixed; real-data results should be re-evaluated with actual checkpoint expert popularity distributions and placement strategies. Flag findings from EP > 1 configurations with this caveat.

Data Schema

Every run produces one row in the results table. Schema is split into four groups.

Table 1 — Configuration (inputs, recorded as metadata)

Column	Type	Values
<code>kernel_config</code>	str	<code>mixtral-7b-dims</code> · <code>mixtral-22b-dims</code> · <code>qwen2-57b-dims</code>
<code>backend</code>	str	<code>DeepEPHT</code> · <code>DeepEPLL</code>
<code>routing_mode</code>	str	<code>uniform</code> · <code>zipfian</code> · <code>random</code> · <code>skewed-2x</code> · <code>skewed-4x</code> · <code>worst-case</code>
<code>zipf_s</code>	float	1.0 default; null if routing_mode ≠ zipfian
<code>num_tokens</code>	int	1 · 4 · 16 · 64 · 256 · 1024 · 4096 · 16384 · 65536 · 131072 · 200000
<code>tp</code>	int	1 · 2 · 4
<code>ep</code>	int	1 · 2 · 4 · 8
<code>seed</code>	int	42 (fixed); recorded so stochastic modes are reproducible
<code>phase</code>	str	A · B · C · D
<code>num_warmup_reps</code>	int	50
<code>num_measured_reps</code>	int	1

Table 2 — Timing outputs (measured, one row per configuration)

Collected via deferred CUDA events; `elapsed_time()` called after a single `synchronize()`.

Column	Type	Unit	Description
<code>latency_mean_ms</code>	float	ms	Mean across 100 reps
<code>latency_p50_ms</code>	float	ms	Median
<code>latency_p95_ms</code>	float	ms	95th percentile — primary SLO proxy
<code>latency_p99_ms</code>	float	ms	99th percentile — tail latency
<code>latency_min_ms</code>	float	ms	Best-case single rep
<code>latency_max_ms</code>	float	ms	Worst-case single rep; flags outliers
<code>latency_std_ms</code>	float	ms	Standard deviation across reps

Table 3 — Derived quantities (computed from Table 1, not measured)

Column	Formula	Unit	Notes
<code>avg_tokens_per_expert</code>	<code>num_tokens × top_k / num_experts</code>	tokens	Global average; ignores EP sharding
<code>avg_tokens_per_expert_per_gpu</code>	<code>avg_tokens_per_expert / ep</code>	tokens	Per-GPU load under EP
<code>expert_GEMM_AMI</code>	<code>2 × avg_tokens_per_expert_per_gpu</code>	flops/byte	Approximation; valid when expert weight bytes dominate
<code>compute_bound_predicted</code>	<code>expert_GEMM_AMI > 295</code>	bool	H100 ridge point ~295; update for other GPU targets
<code>imbalance_ratio_alpha</code>	<code>max_tokens_any_expert / avg_tokens_per_expert</code>	—	Computed from the actual <code>topk_ids</code> tensor after construction, before kernel call

Column	Formula	Unit	Notes
<code>dispatch_bytes_theoretical</code>	<code>num_tokens × hidden_dim × 2</code>	bytes	FP16; bytes sent per GPU in a balanced all-to-all
<code>expert_weight_bytes_per_gpu</code>	<code>(num_experts/ep) × (d × d_ff + d_ff × d) × 2</code>	bytes	W1 + W2 per GPU shard, FP16
<code>tokens_to_hottest_expert</code>	<code>max(expert_token_counts from topk_ids)</code>	tokens	Raw count; used to compute α

Table 4 — NCU hardware counters (subset of runs; EP=1 configurations only for clean attribution)

Run with: `ncu --metrics <metrics> --target-processes all python bench.py`

Column	NCU metric	Unit	Expected value
<code>dram_bytes_read</code>	<code>dram_bytes_read.sum</code>	bytes	—
<code>dram_bytes_write</code>	<code>dram_bytes_write.sum</code>	bytes	—
<code>dram_bytes_theoretical_min</code>	<code>2 × expert_weight_bytes_per_gpu + dispatch_bytes_theoretical</code>	bytes	Lower bound on I/O
<code>dram_efficiency_pct</code>	<code>dram_bytes_theoretical_min / (dram_bytes_read + dram_bytes_write) × 100</code>	%	Higher = less redundant movement
<code>sm_utilization_pct</code>	<code>sm_throughput.avg.pct_of_peak_sustained_elapsed</code>	%	Low in memory-bound regime
<code>l2_hit_rate_pct</code>	<code>l2tex__t_sector_hit_rate.pct</code>	%	Expected near 0% — streaming KV/weight reads
<code>nccl_send_bytes</code>	<code>nccl_send_bytes.sum</code>	bytes	EP > 1 only; null at EP=1
<code>nccl_rcv_bytes</code>	<code>nccl_rcv_bytes.sum</code>	bytes	EP > 1 only; null at EP=1
<code>nccl_bytes_theoretical</code>	<code>num_tokens × hidden_dim × 2 × (ep-1)/ep</code>	bytes	Ring all-to-all expected volume per GPU
<code>nccl_efficiency_pct</code>	<code>nccl_bytes_theoretical / nccl_send_bytes × 100</code>	%	< 100% means excess communication

NCU runs are expensive — limit to: `{mixtral-7b-dims} × {uniform, zipfian} × {64, 1024, 16384} tokens × {tp1-ep1, tp1-ep4}`. This is 24 configurations, sufficient to anchor the AML and streaming behavior claims.

Phase Run Matrices

The four phases differ in which axes are swept and which are held fixed.

Phase A — Routing sensitivity at single GPU (no communication noise)

Fixed: `backend=DeepEPHT`, `tp=1`, `ep=1`

	uniform	zipfian	random	skewed-2x	skewed-4x	worst-case
<code>mixtral-7b-dims</code>	✓ all 11 token values	✓	✓	✓	✓	✓

	uniform	zipfian	random	skewed-2x	skewed-4x	worst-case
mixtral-22b-dims	✓	✓	✓	✓	✓	✓
qwen2-57b-dims	✓	✓	✓	✓	✓	✓

Total: $3 \times 6 \times 11 = 198$ rows

Phase B — Dispatch cost vs EP degree (routing imbalance $\times$ parallelism)

Fixed: num_tokens=128, tp=1; routing reduced to two modes to keep matrix tractable

	ep=1	ep=2	ep=4	ep=8
mixtral-7b-dims $\times$ uniform $\times$ DeepEPHT	✓	✓	✓	✓
mixtral-7b-dims $\times$ uniform $\times$ DeepEPLL	✓	✓	✓	✓
mixtral-7b-dims $\times$ skewed-4x $\times$ DeepEPHT	✓	✓	✓	✓
mixtral-7b-dims $\times$ skewed-4x $\times$ DeepEPLL	✓	✓	✓	✓
mixtral-22b-dims $\times$... (same 4 rows)	✓	✓	✓	✓
qwen2-57b-dims $\times$... (same 4 rows)	✓	✓	✓	✓

Total: 3×2 routing $\times$ 2 backends $\times$ 4 EP = 48 rows

Phase C — Token count $\times$ TP/EP at uniform routing (GEMM crossover mapping)

Fixed: routing_mode=uniform, backend=DeepEPHT

TP/EP config	mixtral-7b-dims	mixtral-22b-dims	qwen2-57b-dims
tp1-ep1	✓ all 11 token values	✓	✓
tp2-ep1	✓	✓	✓
tp1-ep2	✓	✓	✓
tp1-ep4	✓	✓	✓
tp2-ep2	✓	✓	✓
tp1-ep8	✓	✓	✓
tp4-ep2	✓	✓	✓

Total: $7 \times 3 \times 11 = 231$ rows

Phase D — Backend comparison (DeepEPHT vs DeepEPLL)

Fixed: tp=1; routing $\times$ num_tokens grid chosen to straddle the memory-bound/compute-bound boundary and the low/high token regime

	num_tokens=64	num_tokens=256	num_tokens=1024	num_tokens=16384
mixtral-7b-dims $\times$ uniform $\times$ DeepEPHT	✓	✓	✓	✓
mixtral-7b-dims $\times$ uniform $\times$ DeepEPLL	✓	✓	✓	✓
mixtral-7b-dims $\times$ zipfian $\times$ DeepEPHT	✓	✓	✓	✓
mixtral-7b-dims $\times$ zipfian $\times$ DeepEPLL	✓	✓	✓	✓
qwen2-57b-dims $\times$ (same 4 rows)	✓	✓	✓	✓

EP sweep within Phase D: ep $\in \{1, 2, 4, 8\}$

Total: 2 kernel_configs × 2 routing_modes × 2 backends × 4 num_tokens × 4 EP = **128 rows**

Grand total across all phases: ~605 rows (before NCU subset)

Controls

- **Warmup:** 50 steps before measurement; discard first 10 measurement steps.
- **Repetitions:** 100 measurement steps per configuration; report mean ± p95.
- **Weight loading:** weights loaded once, pinned to GPU memory. Do not re-load between configurations.
- **Routing seed:** fix `torch.manual_seed(42)` for stochastic distributions; report seed in results.
- **Precision:** FP16 throughout. Weight tensors and activations both FP16; no mixed-precision unless testing that as a knob explicitly.

Experiment Matrix

Priority order for runs (in case of time/hardware constraints):

1. **Phase A** — Routing distribution × num_tokens at `tp1-ep1` (single GPU): validates harness, establishes imbalance-latency curve with no communication noise. All five routing modes × full token sweep.
2. **Phase B** — Routing distribution × EP degree at fixed num_tokens=128: isolates dispatch cost as EP scales. `uniform` and `skewed-4x` only to keep matrix manageable.
3. **Phase C** — num_tokens × TP/EP config at `uniform` routing: maps the GEMM compute-bound crossover across parallelism configurations.
4. **Phase D** — DeepEPHT vs DeepEPPL: fixed routing × fixed num_tokens, vary EP. Identify where each backend's latency advantage flips (expected: DeepEPPL wins at low num_tokens, DeepEPHT wins at high num_tokens).